

LinkForge Pro

Interview Redeployment Guide — Every step explained with full context

[AWS Lambda](#) [API Gateway](#) [DynamoDB](#) [S3 Static](#) [CloudFormation](#) [Python 3.9](#) [Vanilla JS](#)

CONTENTS

1. [Architecture Overview](#)
2. [Prerequisites & One-time Setup](#)
3. [Phase 1 — Infrastructure \(CloudFormation\)](#)
4. [Phase 2 — Backend \(Lambda Functions\)](#)
5. [Phase 3 — Frontend \(S3 Static Site\)](#)
6. [Verification & Testing](#)
7. [DynamoDB Schema Reference](#)
8. [API Endpoints Reference](#)
9. [Troubleshooting Quick Fixes](#)
10. [Interview Master Checklist](#)
11. [Talking Points for Interviewers](#)

1 Architecture Overview — Understand This Before Touching Anything

WHY READ ARCHITECTURE FIRST?

Every command you run is just making one box in this diagram real. If you understand what each box is and why it connects to the next one, you will never be lost when something fails — you will know exactly which layer to look at.



Component	AWS Service	Purpose	Key Config
Frontend	S3 Static Website	Serve HTML/CSS/JS — no server needed, S3 acts as a web host	Public read, index.html as entry

API Layer	API Gateway (HTTP v2)	Single front door — routes requests to the right Lambda based on URL and method	CORS all origins, 3 routes
Create Link	Lambda Python 3.9	Validates URL, generates/checks short code, writes to DynamoDB	POST /links → 201
Redirect	Lambda Python 3.9	Looks up short code, increments click count, returns 301 redirect	GET /{code} → 301
Admin	Lambda Python 3.9	Scans all DynamoDB records, calculates stats, returns to dashboard	GET /admin/links → 200
Database	DynamoDB on-demand	NoSQL key-value — PK is the short code so every lookup is O(1)	PK = code (String)
IaC	CloudFormation	Defines every resource as YAML — entire infra in one command	infrastructure.yaml
Logs	CloudWatch	Lambda print() output streams here automatically — essential for debugging	14-day retention, 3 groups

 The deployment sequence exists because of hard dependencies:

Phase 1 (CloudFormation) must go first — it creates DynamoDB, the IAM role for Lambda, and the API Gateway shell that Phase 2 needs. Phase 2 (Lambda) must go before Phase 3 because the frontend needs the working API URL. You cannot reverse this order.

2 Prerequisites & One-time Setup

WHY DO THESE BEFORE ANYTHING ELSE?

These are the tools and credentials your local machine needs to talk to AWS and run deployment scripts. Without them, every command in this guide will fail instantly. These are done once per machine — never repeated per project.

1 Install AWS CLI and verify

```
aws --version
# Expected: aws-cli/2.x.x Python/3.x.x ...
```

WHAT IS AWS CLI AND WHY DO YOU NEED IT?

AWS CLI (Command Line Interface) lets you control every AWS service from your terminal instead of clicking through the web console. Every `aws cloudformation deploy`, `aws lambda create-function`, and `aws s3 sync` command in this guide runs through it. Without CLI, you would have to do every step manually in a browser — slow, error-prone, impossible to automate or repeat. Download from aws.amazon.com/cli if not installed.

2 Configure AWS credentials

```
aws configure
# AWS Access Key ID: <your key>
# AWS Secret Access Key: <your secret>
# Default region name: us-east-1
# Default output format: json

# Always verify after configuring:
aws sts get-caller-identity
```

WHAT DOES AWS CONFIGURE DO?

Stores your Access Key ID and Secret in `~/.aws/credentials`. Every CLI command after this automatically uses those credentials — you never type them per command. Setting **us-east-1 as default region** is important because all resources in this project (DynamoDB, Lambda, API Gateway, S3) are hardcoded to us-east-1. **Output json** makes CLI responses structured JSON, easier to parse with `--query` and `python3 -m json.tool`.

WHAT DOES AWS STS GET-CALLER-IDENTITY DO AND WHY RUN IT?

STS = Security Token Service. This asks AWS "who am I?" and returns your Account ID, IAM User ARN, and User ID. It confirms credentials work before you touch anything. If this fails, nothing else will work. You also need the 12-digit **Account ID** it returns later when constructing Lambda ARNs manually in Phase 2.

⚠ IAM permissions required on your AWS user:

CloudFormation (full), Lambda (full), API Gateway (full), DynamoDB (full), S3 (full), IAM — create/update roles and policies, CloudWatch Logs — create/write. In an interview environment these are usually pre-granted.

3 Verify Node.js and Python versions

```
node --version    # Must be v16+
python3 --version # Must be 3.9+
```

WHY DO YOU NEED BOTH?

Python 3.9 is the Lambda runtime — your three .py files run as Python inside AWS Lambda. You also need it locally to run `deploy-backend.py`. **Node.js 16+** runs the `deploy-frontend.js` script that syncs files to S3 and updates the frontend config. It also powers the

npm scripts in package.json.

4 Clone repo and install dependencies

```
git clone https://github.com/yourusername/linkforge-pro.git
cd linkforge-pro
npm install
pip3 install boto3
```

WHAT DOES EACH COMMAND DO?

git clone — downloads the entire project: infrastructure.yaml, all three Lambda .py files, the frontend HTML/JS/CSS, and all deploy scripts. **cd linkforge-pro** — critical, all relative paths in subsequent commands (like infrastructure/infrastructure.yaml) assume you are in this root folder. **npm install** — installs Node.js dev dependencies from package.json, enables the npm run shortcuts. **pip3 install boto3** — boto3 is the official AWS SDK for Python. Lambda functions use it to talk to DynamoDB (`boto3.resource('dynamodb')`). Also needed locally for deploy-backend.py which calls the Lambda API using boto3.

3 Phase 1 — Infrastructure via CloudFormation

Why Phase 1 must run first: CloudFormation creates DynamoDB (Lambda must write somewhere), the IAM role (Lambda must have permissions), API Gateway (Lambda needs a front door), and the S3 bucket (frontend needs a home). None of Phase 2 or Phase 3 is possible without Phase 1. This is a hard dependency chain, not a preference.

What CloudFormation actually is:

Instead of clicking "Create Table" in DynamoDB, "Create API" in API Gateway, etc. manually, you write one YAML file describing every resource. CloudFormation reads it, figures out the creation order based on dependencies, and creates everything. This is Infrastructure as Code (IaC). Benefit: delete and recreate the entire stack identically with one command. No forgetting a setting, no manual clicking.

1 Deploy the CloudFormation stack

```
aws cloudformation deploy \
  --template-file infrastructure/infrastructure.yaml \
```

```
--stack-name url-shortener-infra \  
--capabilities CAPABILITY_NAMED_IAM \  
--region us-east-1 \  
--parameter-overrides Environment=dev \  
--no-fail-on-empty-changeset
```

Flag / Argument

aws cloudformation deploy

What it does and why it is here

Creates or updates a CloudFormation stack. Internally it creates a "change set" (a diff of what will change), shows you, then executes it. If the stack does not exist yet, it creates everything fresh. If it exists, it updates only what changed — safe to run multiple times.

--template-file infrastructure/infrastructure.yaml

Points to the IaC YAML file relative to your current directory. This file defines every resource: DynamoDB table, API Gateway HTTP API, Lambda IAM role, S3 bucket, S3 bucket policy, and 3 CloudWatch Log Groups. CloudFormation reads it and figures out creation order automatically.

--stack-name url-shortener-infra

The name CloudFormation uses to group all your resources. Think of it as a project folder — all resources created by this template are tagged with this stack name. You reference this same name in every later command to get outputs (API URL, bucket name, etc.). Must be unique in your AWS account + region combination.

--capabilities CAPABILITY_NAMED_IAM

An explicit safety acknowledgement required whenever your template creates IAM resources with hardcoded names. Your template has RoleName: url-shortener-infra-lambda-role — an explicitly named IAM role. AWS forces you to acknowledge this because named IAM resources are harder to track and audit. CAPABILITY_IAM (without NAMED) only covers auto-named IAM resources and will fail here.

--region us-east-1

Tells CloudFormation which AWS region to deploy into. All resources will be created in us-east-1 (N. Virginia). The backend Lambda code is also hardcoded to us-east-1 (the DynamoDB client is initialised with region_name='us-east-1'). These must match or Lambda cannot reach DynamoDB.

`--parameter-overrides Environment=dev`

Passes a value into the template Parameters section. The template uses this as the API Gateway stage name, which becomes the /dev path segment in your API URL (<https://id.execute-api.region.amazonaws.com/dev>). If you passed `Environment=prod`, the stage would be /prod.

`--no-fail-on-empty-changeset`

Normally, if nothing changed in the template and you re-run this command, CloudFormation returns an error saying "no changes." This flag suppresses that error so re-running is always safe — essential in scripts and CI/CD pipelines where you want to guarantee the stack is in sync without checking first.

WHAT HAPPENS INSIDE AWS WHEN YOU RUN THIS COMMAND?

CloudFormation reads your YAML, builds a dependency graph (e.g. S3 BucketPolicy depends on S3 Bucket, Lambda Role depends on nothing), then creates resources in the right order — parallelising where safe. Each resource shows "CREATE_IN_PROGRESS" then "CREATE_COMPLETE". The whole stack takes 2–3 minutes. Resources created: LinksTable (DynamoDB), HttpApi (API Gateway v2), ApiStage (/dev stage), LambdaExecutionRole (IAM), FrontendBucket (S3), FrontendBucketPolicy (public read), and 3 CloudWatch Log Groups.

Issue hit during deployment: `InsufficientCapabilitiesException`

An error occurred (`InsufficientCapabilitiesException`) when calling the `CreateChangeSet` operation: Requires capabilities : [`CAPABILITY_NAMED_IAM`]

Why it happened:

The original command used `--capabilities CAPABILITY_IAM`. But the template has `RoleName: !Sub "${AWS::StackName}-lambda-role"` — an IAM role with an explicit custom name. AWS requires the stricter `CAPABILITY_NAMED_IAM` for any template that names IAM resources explicitly.

Fix:

Replace `CAPABILITY_IAM` with `CAPABILITY_NAMED_IAM` (already corrected in the command above).

Rule to remember:

`CAPABILITY_IAM` = unnamed/auto-named IAM resources OK. `CAPABILITY_NAMED_IAM` = explicitly-named IAM resources OK (superset — covers both cases). When in doubt, always use `CAPABILITY_NAMED_IAM`. It will never fail where `CAPABILITY_IAM` would succeed.

WHY IS THIS STEP 1 OF THE ENTIRE DEPLOYMENT?

Because every other step in this guide depends on what this command creates. Lambda

create-function needs the IAM role ARN (created here). API Gateway route creation needs the API ID (created here). S3 sync needs the bucket name (created here). Lambda needs a DynamoDB table to write to (created here). This is the foundation — skip or fail this and nothing else in the guide is possible.

2 Get and save the stack outputs

```
aws cloudformation describe-stacks \
  --stack-name url-shortener-infra \
  --query 'Stacks[0].Outputs[*].[OutputKey,OutputValue]' \
  --output table
```

Flag / Argument

describe-stacks

What it does and why

Fetches the current state of a named CloudFormation stack — its status, parameters, and importantly its Outputs. Outputs are values the template explicitly exports (API URL, bucket name, etc.) that you need to use in subsequent steps. Without this command you would have to hunt through the AWS console to find each value.

--stack-name url-shortener-infra

Must exactly match the name used in the deploy command. This identifies which stack's data to return.

--query 'Stacks[0].Outputs[*].[OutputKey,OutputValue]'

JMESPath query that filters the large JSON response. Stacks[0] = first result (only one stack with this name). Outputs[*] = all outputs. . [OutputKey,OutputValue] = return only the name and value columns, dropping extra metadata. Without this, the full response is hard to read.

--output table

Formats the filtered result as a human-readable ASCII table. Much easier to scan than raw JSON, especially when you are looking for specific values to copy into other commands.

WHAT OUTPUTS YOU WILL SEE AND WHY YOU NEED EACH ONE

ApiGatewayUrl — e.g. `https://abc123.execute-api.us-east-1.amazonaws.com/dev`. This goes into `app.js` and `admin.js` in Phase 3. Without it the frontend will not talk to your backend.

ApiGatewayId — e.g. `abc123`. Needed in Phase 2 to create integrations and routes, and to grant Lambda permissions.

DynamoDBTableName — the actual table name (`url-shortener-infra-links`). Your Lambda functions have this hardcoded — verify they match exactly.

LambdaRoleArn — the full ARN of the IAM role. You supply this when creating each Lambda function.

S3BucketName — the actual bucket name with account ID suffix. Needed in Phase 3 for the sync command.

S3WebsiteUrl — the public HTTP URL of your frontend. This is what you open in the browser.

WHY COLLECT OUTPUTS IMMEDIATELY AFTER PHASE 1?

Every step in Phase 2 and Phase 3 requires values that only exist after Phase 1. Collecting them now in one command means you have everything at hand before starting Phase 2 — no stopping mid-deployment to look things up. In a timed interview, this saves valuable minutes.

3 Verify each resource was created

```
# Verify DynamoDB table
aws dynamodb list-tables --region us-east-1

# Verify S3 bucket
aws s3 ls | grep url-shortener

# Verify API Gateway
aws apigatewayv2 get-apis --region us-east-1
```

WHY VERIFY MANUALLY INSTEAD OF TRUSTING CLOUDFORMATION SAID "CREATE_COMPLETE"?

CloudFormation reports completion for the stack as a whole, but occasionally resources have subtle issues — wrong billing mode, wrong region, missing policy. Verifying takes 30 seconds and catches issues before you build on top of them. If you skip this and something is wrong, you will get confusing errors deep in Phase 2 and waste time debugging the wrong layer.

aws dynamodb list-tables — confirms the exact table name. Lambda hardcodes `url-shortener-infra-links`. If CloudFormation named it differently (e.g. because of a stack name mismatch), every Lambda DynamoDB call fails with `ResourceNotFoundException`.

aws apigatewayv2 get-apis — note the v2 here. This project uses API Gateway HTTP API (v2), not REST API (v1). They are different services with different CLI commands. If you run `aws apigateway get-rest-apis` (v1) you will see nothing — use `apigatewayv2`.

4 Phase 2 — Backend: Deploy 3 Lambda Functions

Why Phase 2 runs second: Lambda functions need the IAM role from Phase 1 to be created with. They need to be attached to the API Gateway from Phase 1 via integrations and routes. You also need the DynamoDB table name from Phase 1 to pass as an environment variable. None of this is possible before Phase 1. Phase 2 runs before Phase 3 because the frontend needs a working API URL — which only makes sense once Lambda functions are wired up.

⚡ What deploying a Lambda function means:

Lambda does not run on a server that is always on. You upload a ZIP file containing your Python code to AWS. AWS stores it. When an API Gateway request arrives, AWS spins up a micro-container, runs your handler function, returns the response, then shuts down. The ZIP is your deployment package. No provisioning, no SSH, no server management.

1 Create the deployment ZIP packages

```
mkdir -p build
```

```
cd src/backend/functions && zip ../../build/create_link.zip create_link.py && cd ../../  
cd src/backend/functions && zip ../../build/redirect_link.zip redirect_link.py && cd ../  
cd src/backend/functions && zip ../../build/admin_links.zip admin_links.py && cd ../../
```

Part

mkdir -p build

What it does and why

Creates a /build directory at the project root for the ZIP files. The -p flag means "create parent directories if needed, and do not error if already exists." Done first because the zip command will fail if the destination directory does not exist.

cd src/backend/functions

Navigates into the folder containing the .py files before zipping. This is critical: when zip runs, the file is added to the archive relative to the current directory. If you zip from elsewhere, the file ends up inside a subfolder in the ZIP, and Lambda will fail to find the handler file at runtime because it looks for it at the root of the ZIP.

zip ../../build/create_link.zip create_link.py

Creates the ZIP archive. ../../build/ navigates three levels up (functions →

backend → src → project root) then into build/. The result is a ZIP where create_link.py sits at the root — exactly how Lambda requires it.

```
cd ../../..
```

Navigates back to the project root after each zip so the next cd command starts from the correct location. Forget this and the paths cascade incorrectly.

WHY ZIP? WHY NOT JUST POINT LAMBDA AT THE .PY FILE DIRECTLY?

Lambda does not have access to your local filesystem or GitHub. The only way to get code into Lambda is to package it as a ZIP and upload it. AWS stores the ZIP, extracts it into a temporary execution environment, and calls your handler when a request arrives. If your function needed external packages like `requests`, you would also run `pip install requests -t .` inside the functions folder to bundle them into the ZIP. This project only uses boto3 which is pre-installed in all Lambda Python runtimes — so the .py file alone is sufficient.

WHY CREATE ALL THREE ZIPS BEFORE CREATING ANY LAMBDA FUNCTION?

The `aws lambda create-function` command requires you to supply the ZIP at creation time. Creating all ZIPs first means you can run all three create-function commands back-to-back without switching tasks. It also makes it easy to spot if a ZIP is missing before wasting time on the create-function calls.

2 Get the Lambda IAM Role ARN

```
ROLE_ARN=$(aws cloudformation describe-stacks \
  --stack-name url-shortener-infra \
  --query "Stacks[0].Outputs[?OutputKey=='LambdaRoleArn'].OutputValue" \
  --output text)

echo "Role ARN: $ROLE_ARN"
# Should print: arn:aws:iam::264449293739:role/url-shortener-infra-lambda-role
```

WHAT IS AN IAM ROLE AND WHY DOES LAMBDA NEED ONE?

An IAM Role is a set of permissions that an AWS service can assume. Your Lambda function needs two things: (1) permission to write logs to CloudWatch so you can debug it, and (2) permission to read and write records in DynamoDB. Without a role granting these, Lambda gets AccessDeniedException on every DynamoDB call. The role was created in Phase 1 with exactly these policies. You are fetching its ARN (Amazon Resource Name — a globally unique identifier for any AWS resource) to assign it to your functions.

COMMAND BREAKDOWN — THE SHELL SUBSTITUTION AND QUERY SYNTAX

`${...}` = shell command substitution. Runs the inner command and stores its printed output in the `ROLE_ARN` variable. Every subsequent reference to `$ROLE_ARN` uses this stored value.

`Stacks[0].Outputs[?OutputKey=='LambdaRoleArn'].OutputValue` = JMESPath filter. From the `Outputs` array, find the item where `OutputKey` equals `'LambdaRoleArn'`, then return only its `OutputValue`. `--output text` returns it as plain text without JSON quotes so it stores cleanly in a shell variable.

WHY FETCH THIS DYNAMICALLY INSTEAD OF HARDCODING THE ARN?

The ARN contains your AWS account ID (12-digit number) and the stack name — both of which differ per person and per environment. Fetching it dynamically means this guide works for any AWS account without edits. Hardcoding would break for anyone using a different account or stack name.

3 Create all 3 Lambda functions

```
# 1. Create Link Function
aws lambda create-function \
  --function-name url-shortener-infra-createLink \
  --runtime python3.9 \
  --role $ROLE_ARN \
  --handler create_link.lambda_handler \
  --zip-file fileb://build/create_link.zip \
  --region us-east-1 \
  --environment Variables={TABLE_NAME=url-shortener-infra-links}

# 2. Redirect Link Function
aws lambda create-function \
  --function-name url-shortener-infra-redirectLink \
  --runtime python3.9 \
  --role $ROLE_ARN \
  --handler redirect_link.lambda_handler \
  --zip-file fileb://build/redirect_link.zip \
  --region us-east-1 \
  --environment Variables={TABLE_NAME=url-shortener-infra-links}

# 3. Admin Links Function
aws lambda create-function \
  --function-name url-shortener-infra-adminLinks \
  --runtime python3.9 \
  --role $ROLE_ARN \
  --handler admin_links.lambda_handler \
  --zip-file fileb://build/admin_links.zip \
  --region us-east-1 \
  --environment Variables={TABLE_NAME=url-shortener-infra-links}
```

Flag

What it does and why set this way

`--function-name`

The name of the Lambda function in AWS. Must be

	unique within a region. Prefixed with the stack name (url-shortener-infra-) to group related functions and avoid collisions with other projects in the same account. The deploy-backend.py script searches for functions matching this prefix pattern.
--runtime python3.9	Tells Lambda which language environment to execute your code in. Must match what your code was written for. Python 3.9 is the version specified in package.json and is stable with good boto3 support. Lambda has boto3 pre-installed for all Python runtimes so you do not need to bundle it in the ZIP.
--role \$ROLE_ARN	Assigns the IAM role fetched in Step 2. This is what gives the function permission to access DynamoDB and write to CloudWatch Logs. Without this, every DynamoDB call returns AccessDeniedException and you would see nothing in CloudWatch either.
--handler create_link.lambda_handler	Tells Lambda exactly which function to call when triggered. Format: filename.function_name. create_link = the Python file (create_link.py, without the .py extension). lambda_handler = the function defined as def lambda_handler(event, context) inside that file. This is the standard Lambda entry point convention — all three files follow it.
--zip-file fileb://build/create_link.zip	The deployment package path. fileb:// prefix means "file, binary mode" — required for binary files like ZIPs (as opposed to text files which use file://). Relative to your current working directory — another reason you must run commands from the project root.
--environment Variables={TABLE_NAME=...}	Sets environment variables inside the Lambda execution environment. Your Python code reads the table name with os.environ.get('TABLE_NAME') rather than hardcoding it. This means you can deploy to a staging environment with a different table just by changing the env var — no code changes needed.

FOR RE-DEPLOYMENTS — USE UPDATE-FUNCTION-CODE INSTEAD

create-function fails if the function already exists (it is not idempotent — cannot be run twice safely). update-function-code replaces the code ZIP without touching configuration (role, timeout, env vars). Use this on every re-deploy after the first:

```
aws lambda update-function-code \
  --function-name url-shortener-infra-createLink \
  --zip-file fileb://build/create_link.zip \
  --region us-east-1
# Repeat for redirectLink and adminLinks
```

4 Wire Lambda functions to API Gateway routes

```
# Get API Gateway ID and build Lambda ARNs
API_ID=$(aws cloudformation describe-stacks \
  --stack-name url-shortener-infra \
  --query "Stacks[0].Outputs[?OutputKey=='ApiGatewayId'].OutputValue" \
  --output text)

ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
REGION=us-east-1
CREATE_ARN="arn:aws:lambda:$REGION:$ACCOUNT_ID:function:url-shortener-infra-createLink"
REDIRECT_ARN="arn:aws:lambda:$REGION:$ACCOUNT_ID:function:url-shortener-infra-redirectLink"
ADMIN_ARN="arn:aws:lambda:$REGION:$ACCOUNT_ID:function:url-shortener-infra-adminLinks"

# Create integrations (the connection objects between API GW and Lambda)
CREATE_INTEG=$(aws apigatewayv2 create-integration \
  --api-id $API_ID --integration-type AWS_PROXY \
  --integration-uri $CREATE_ARN --payload-format-version 2.0 \
  --query IntegrationId --output text)

REDIRECT_INTEG=$(aws apigatewayv2 create-integration \
  --api-id $API_ID --integration-type AWS_PROXY \
  --integration-uri $REDIRECT_ARN --payload-format-version 2.0 \
  --query IntegrationId --output text)

ADMIN_INTEG=$(aws apigatewayv2 create-integration \
  --api-id $API_ID --integration-type AWS_PROXY \
  --integration-uri $ADMIN_ARN --payload-format-version 2.0 \
  --query IntegrationId --output text)

# Create the actual HTTP routes pointing to each integration
aws apigatewayv2 create-route --api-id $API_ID \
  --route-key "POST /links" --target integrations/$CREATE_INTEG

aws apigatewayv2 create-route --api-id $API_ID \
  --route-key "GET /{code}" --target integrations/$REDIRECT_INTEG

aws apigatewayv2 create-route --api-id $API_ID \
  --route-key "GET /admin/links" --target integrations/$ADMIN_INTEG
```

WHY IS WIRING ROUTES A SEPARATE STEP FROM CREATING LAMBDA FUNCTIONS?

API Gateway and Lambda are two completely separate AWS services. Creating a Lambda function does NOT automatically make it reachable via HTTP. You need to explicitly tell API Gateway: "when you receive a POST /links request, forward it to this Lambda." That connection is called an integration. Think of it like this: Lambda = a phone. Creating Lambda = buying the phone. Creating an integration and route = assigning that phone a public number and advertising it.

⚠️ Critical lesson learned in testing: Do NOT use this manual wiring approach with a CloudFormation-managed API Gateway

Manually creating integrations and routes on an API Gateway that was created by CloudFormation is a fundamental conflict. The CloudFormation-managed stage tracks a "deployment snapshot" of the time it was deployed. Routes added manually afterwards are NOT included in that snapshot. The stage ignores them and continues serving the old deployment (which has no routes). The result is persistent 500 errors where Lambda is never invoked, even though the routes visually appear when you run `get-routes`.

The correct approach for this project:

Lambda functions, integrations, and routes must ALL be defined inside `infrastructure.yaml`. Since CloudFormation manages them together and deploys them atomically. Do not create any of them manually as a separate step. When everything is in the YAML, CloudFormation creates them with deployment tracking from the start.

If you already created resources manually and need to fix it:

```
# 1. Delete manually created routes (integrations cannot be deleted while routes reference them)
aws apigatewayv2 delete-route --api-id $API_ID --route-id <route-id> --region $REGION

# 2. Delete manually created integrations
aws apigatewayv2 delete-integration --api-id $API_ID --integration-id <integ-id> --region $REGION

# 3. Delete manually created Lambda functions
aws lambda delete-function --function-name url-shortener-infra-createLink --region us-east-1
aws lambda delete-function --function-name url-shortener-infra-redirectLink --region us-east-1
aws lambda delete-function --function-name url-shortener-infra-adminLinks --region us-east-1

# 4. Add Lambda + integrations + routes to infrastructure.yaml, then redeploy:
aws cloudformation deploy \
  --template-file infrastructure/infrastructure.yaml \
  --stack-name url-shortener-infra \
  --capabilities CAPABILITY_NAMED_IAM \
  --region us-east-1 --parameter-overrides Environment=dev

# 5. After stack deploys, upload real code (CloudFormation uses placeholder):
aws lambda update-function-code --function-name url-shortener-infra-createLink \
  --zip-file fileb://build/create_link.zip --region us-east-1
aws lambda update-function-code --function-name url-shortener-infra-redirectLink \
  --zip-file fileb://build/redirect_link.zip --region us-east-1
aws lambda update-function-code --function-name url-shortener-infra-adminLinks \
  --zip-file fileb://build/admin_links.zip --region us-east-1
```

Concept	Explanation
create-integration	Creates a connection object between API Gateway and a Lambda. <code>AWS_PROXY</code> type means API Gateway passes the full HTTP request (headers, body, path params, method) directly to Lambda as a JSON event object, and Lambda returns the full HTTP response including status code and headers. Lambda handles everything — this is the standard pattern.
--payload-format-version 2.0	Specifies the format of the event object Lambda receives.

Version 2.0 is simpler and newer. Your Lambda code uses the v2.0 structure (it checks `event.get('requestContext', {}).get('http', {}).get('method')`). Using 1.0 would break the CORS handling logic in all three functions because the event structure is different.

--route-key "GET /{code}"

The HTTP method + path pattern that triggers this route. Curly braces denote a path parameter — `{code}` captures whatever comes after the slash and passes it as `pathParameters.code` in the Lambda event. This is how `redirect_link.py` gets the short code from the URL.

--target integrations/\$INTEG_ID

Links the route to the integration just created. The "integrations/" prefix is required — it is how API Gateway distinguishes integration targets from other target types. The integration ID was captured in the variable by storing the create-integration output.

5 Grant API Gateway permission to invoke Lambda

```
aws lambda add-permission \  
  --function-name url-shortener-infra-createLink \  
  --statement-id apigw-create \  
  --action lambda:InvokeFunction \  
  --principal apigateway.amazonaws.com \  
  --source-arn "arn:aws:execute-api:$REGION:$ACCOUNT_ID:$API_ID/*/*"  
  
# Repeat for redirectLink with --statement-id apigw-redirect  
# Repeat for adminLinks with --statement-id apigw-admin
```

WHY IS THIS A SEPARATE STEP? DID CREATING THE INTEGRATION NOT WIRE THINGS UP?

This is a common point of confusion. Creating the integration (Step 4) tells API Gateway which Lambda to call. But Lambda has its own separate resource-based permission system — a policy attached to the function itself — that controls who is allowed to invoke it. By default, nothing is allowed to invoke Lambda, not even API Gateway. You must explicitly grant the permission here. This is AWS defense-in-depth security: two separate checks must both pass. API Gateway must be allowed to invoke Lambda (this step) AND Lambda must have an IAM role to call DynamoDB (Phase 1). Two separate permission layers.

Flag

--statement-id apigw-create

What it does

A unique label for this permission entry in the Lambda resource policy. Must be unique per function. If you run this command twice with the same statement-id it fails with "already exists" — use a different ID or delete the existing permission first.

--action lambda:InvokeFunction

The specific IAM action being granted — the ability

<code>--principal apigateway.amazonaws.com</code>	Who is being granted permission — the API Gateway service itself, identified by its service principal. This is not a user or role — it is AWS's own service identity.
<code>--source-arn "arn:aws:execute-api:..."</code>	Scopes the permission so ONLY your specific API Gateway (identified by \$API_ID) can invoke this Lambda. The /* /* at the end means any stage, any method. Without this, any API Gateway in your entire account could invoke this function — a security risk.

5 Phase 3 — Frontend: Deploy to S3

Why Phase 3 runs last: The JavaScript files must contain your real API Gateway URL before being uploaded. That URL only exists after Phase 1 (API Gateway created) and is only meaningful after Phase 2 (Lambda functions wired up and responding to requests). Deploying frontend first would mean serving a broken app that points nowhere.

🌐 How S3 static hosting works:

S3 is normally cloud file storage. In "static website" mode it acts as a web server — serving HTML/CSS/JS directly to browsers over HTTP. No Node.js process, no nginx, no server to manage. Files are stored as S3 objects and served at a public URL. Scales to millions of requests automatically. Extremely cheap — you pay for storage and data transfer only.

1 Update the API URL in frontend JS files

```
# First get your real API URL
API_URL=$(aws cloudformation describe-stacks \
  --stack-name url-shortener-infra \
  --query "Stacks[0].Outputs[?OutputKey=='ApiGatewayUrl'].OutputValue" \
  --output text)
echo "Your API URL: $API_URL"
```

Now open `src/frontend/assets/app.js` AND `src/frontend/assets/admin.js` and update the CONFIG at the top of each:

```
const CONFIG = {
  API_BASE_URL: 'https://<YOUR_API_ID>.execute-api.us-east-1.amazonaws.com/dev',
  DEMO_MODE: false,    // ← must be false for production
  AUTO_FALLBACK: true
};
```


WHY MUST YOU UPDATE THIS BEFORE UPLOADING TO S3?

The frontend JavaScript runs entirely in the user's browser — it is not server-side code. When a user clicks "Shorten URL", their browser directly calls the `API_BASE_URL` via JavaScript `fetch()`. If this URL is wrong, the app detects the failure and silently falls back to `DEMO_MODE` — showing fake locally-generated data with no real backend interaction. Your links would not actually be stored anywhere.

You must update **both `app.js`** (used by `index.html` for the shortener) and **`admin.js`** (used by `admin.html` for the dashboard) — they each have their own `CONFIG` object at the top.

Setting `DEMO_MODE: false` forces the app to call the real API instead of generating mock data locally.

WHY EDIT FILES LOCALLY BEFORE UPLOADING RATHER THAN AFTER?

S3 serves static files — there is no way to inject runtime variables or run code on S3. The API URL must be baked into the JS file before it is uploaded. This is the standard pattern for static web apps. Modern frameworks like React and Vue use build-time environment variables (`process.env.REACT_APP_API_URL`) for exactly the same reason — the URL gets embedded into the built bundle before deployment.

2 Get the S3 bucket name

```
BUCKET=$(aws cloudformation describe-stacks \
  --stack-name url-shortener-infra \
  --query "Stacks[0].Outputs[?OutputKey=='S3BucketName'].OutputValue" \
  --output text)
echo "Bucket: $BUCKET"
```

WHY NOT JUST TYPE THE BUCKET NAME DIRECTLY?

The bucket name includes your AWS account ID as a suffix (e.g. `url-shortener-infra-frontend-264449293739`). S3 bucket names must be globally unique across every AWS account in the world. CloudFormation appends your account ID using `${AWS::AccountId}` to guarantee uniqueness. Since the account ID differs per person, fetching the name dynamically is the safe approach.

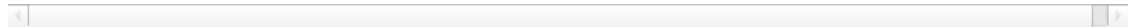
3 Sync frontend files to S3

```
# Upload all files
aws s3 sync src/frontend/ s3://$BUCKET/ \
  --region us-east-1 --delete --exclude "*.md"
```

```
# Fix content types (so browsers handle files correctly)
aws s3 cp s3://$BUCKET/ s3://$BUCKET/ \
  --recursive --exclude "*" --include "*.html" \
  --content-type "text/html" --metadata-directive REPLACE --region us-east-1

aws s3 cp s3://$BUCKET/ s3://$BUCKET/ \
  --recursive --exclude "*" --include "*.js" \
  --content-type "application/javascript" --metadata-directive REPLACE --region us-east-1

aws s3 cp s3://$BUCKET/ s3://$BUCKET/ \
  --recursive --exclude "*" --include "*.css" \
  --content-type "text/css" --metadata-directive REPLACE --region us-east-1
```



Flag / Part	What it does and why
aws s3 sync	Compares local files with what is in S3 and only uploads new or changed files (like rsync). Much faster than re-uploading everything on re-deploys. Without --delete, files removed locally stay in S3 forever.
src/frontend/	Local source directory containing index.html, admin.html, assets/app.js, assets/admin.js, assets/styles.css. Everything here gets mirrored to S3 at the same relative path.
--delete	Removes objects from S3 that no longer exist locally. Without this, old/renamed files accumulate. If you renamed index.html to app.html, the old index.html would stay in S3 serving stale content.
--exclude "*.md"	Skips markdown files like README.md. Developer documentation — no reason to serve them publicly from a website bucket.
Content-type correction commands	When aws s3 sync uploads files, it sometimes guesses MIME types incorrectly or defaults to application/octet-stream. Browsers use the Content-Type HTTP header to decide how to handle a file. If .js files have the wrong content type, modern browsers refuse to execute them as scripts (security policy). These cp commands overwrite the content-type metadata on already-uploaded files.
--metadata-directive REPLACE	When copying a file to the same location (same source and destination — used here only to update metadata), S3 normally refuses to overwrite metadata. REPLACE forces it to do so. Without this flag, the content-type corrections silently do nothing.

4 Get your live URLs

```
aws cloudformation describe-stacks \
  --stack-name url-shortener-infra \
  --query "Stacks[0].Outputs[?OutputKey=='S3WebsiteUrl'].OutputValue" \
```

--output text

```
# Main app:      http://<bucket>.s3-website-us-east-1.amazonaws.com
# Admin panel:  http://<bucket>.s3-website-us-east-1.amazonaws.com/admin.html
```

WHY HTTP NOT HTTPS?

S3 static website hosting only supports HTTP. To get HTTPS you would add CloudFront (AWS CDN) in front of S3 and attach an SSL certificate from ACM (AWS Certificate Manager). That is on the project roadmap but not implemented in this version. For an interview demo, HTTP is fine. Worth mentioning as a "what I would add next" when asked.

One-command deploy after the first manual deployment:

```
chmod +x scripts/deploy/*.sh
./scripts/deploy/deploy-all.sh production
```

Runs Phase 1 → 2 → 3 automatically and prints final URLs. Use for re-deploys once you have done the full manual walk-through at least once.

6 Verification & Testing — Confirm Every Layer Works

WHY TEST EACH LAYER INDIVIDUALLY BEFORE CHECKING THE UI?

If you open the browser and something looks broken, you will not know whether the problem is in frontend JS, API Gateway routing, Lambda code, or DynamoDB access. Testing from the bottom up (database → Lambda → API → UI) means when you find a failure you know exactly which layer to fix. In an interview, methodical debugging like this looks far better than random clicking.

1 Test Create Link endpoint

```
API_URL="https://<your-api-id>.execute-api.us-east-1.amazonaws.com/dev"

curl -X POST $API_URL/links \
  -H "Content-Type: application/json" \
  -d '{"url": "https://google.com", "customCode": "testlink"}' \
  | python3 -m json.tool
```

WHAT TO EXPECT AND WHAT EACH PART IS TESTING

Expected: `{"success": true, "shortCode": "testlink", "shortUrl": "...", "targetUrl":`

```
"...", "createdAt": "..."}]
```

`curl -X POST` sends an HTTP POST. `-H "Content-Type: application/json"` tells API Gateway to parse the body as JSON. `-d '{...}'` is the request body. `| python3 -m json.tool` pretty-prints the JSON response.

`{"message": "Not Found"}` → POST /links route does not exist — Phase 2 Step 4 was not completed. 403 Forbidden → Lambda invoke permission not granted — Phase 2 Step 5 missing. 500 Internal Server Error → Lambda ran but threw an exception — check CloudWatch Logs for the traceback.

Issue 1 Hit: {"message": "Internal Server Error"} — 500 with empty CloudWatch logs

```
{"message": "Internal Server Error"}
```

```
# Checking logs showed nothing:
aws logs describe-log-streams \
  --log-group-name /aws/lambda/url-shortener-infra-createLink \
  --region us-east-1 --order-by LastEventTime --descending --max-items 1
# → logStreams: [] ← Lambda was never invoked at all
```

What empty logStreams means:

If Lambda was invoked and crashed, you would see a log stream. An empty logStreams list means the request was never processed by Lambda — the 500 is coming from API Gateway itself, before it even reaches Lambda. The request was rejected by the API Gateway layer.

Root cause discovered:

The IntegrationUri ARNs were malformed — `:function:` got cut off when the ARN variable was constrained to a maximum length. This resulted in broken ARNs like `arn:aws:lambda:us-east-1:690081480550:function:url-shortener-infra-createLink` instead of `arn:aws:lambda:us-east-1:690081480550:function:url-shortener-infra-createLink`. API Gateway rejected the request with a 500.

How to diagnose a 500 systematically:

```
# Step 1: Check if Lambda is even being invoked
aws logs describe-log-streams \
  --log-group-name /aws/lambda/url-shortener-infra-createLink \
  --region us-east-1 --order-by LastEventTime --descending --max-items 1

# Step 2: If logStreams is empty, bypass API Gateway and invoke Lambda directly
aws lambda invoke \
  --function-name url-shortener-infra-createLink \
  --region us-east-1 \
  --payload '{"body": "{\"url\": \"https://google.com\"}", "requestContext": {"http": {"method": "POST", "path": "/links", "protocol": "HTTP/1.1", "source_ip": "192.168.1.1", "user_agent": "curl/7.28.1"}}}' \
  --cli-binary-format raw-in-base64-out \
  response.json && cat response.json
# If this returns 201: Lambda code is fine, problem is API Gateway config
# If this fails: Lambda code has a bug

# Step 3: Verify integration URIs are correct
aws apigatewayv2 get-integrations --api-id $API_ID --region us-east-1
# Check that IntegrationUri contains :function: not a garbled string
```

🔴 Issue 2 Hit: Malformed Lambda ARN in IntegrationUri — :function: missing

```
# Broken (what was found):
"IntegrationUri": "arn:aws:lambda:us-east-1:690081480550nction:url-shortener-infra-createLink"

# Correct (what it should be):
"IntegrationUri": "arn:aws:lambda:us-east-1:690081480550:function:url-shortener-infra-createLink"
```

Why it happened:

When constructing the ARN string manually in the terminal, if the variable `$ACCOUNT_ID` is not a trailing character issue, the shell concatenates it directly into `:function:` and the `:f` prefix is absorbed. The ARN looks almost right but is invalid — API Gateway silently returns 500 rather than a clear "bad ARN" error.

Fix — delete routes first (routes block integration deletion), then delete integration and recreate with correct ARNs:

```
# You cannot delete integrations while routes reference them
# Must delete routes first:
aws apigatewayv2 delete-route --api-id $API_ID --route-id <route-id> --region $REGION
# Repeat for all 3 routes

# Then delete the broken integrations:
aws apigatewayv2 delete-integration --api-id $API_ID --integration-id <integ-id> --region $REGION
# Repeat for all 3 integrations

# Then build ARNs correctly – verify $ACCOUNT_ID is set first:
echo $ACCOUNT_ID # Must print a 12-digit number, e.g. 690081480550
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text) # Re-set if empty

CREATE_ARN="arn:aws:lambda:$REGION:$ACCOUNT_ID:function:url-shortener-infra-createLink"
echo $CREATE_ARN # Verify it looks correct before using it
```

Prevention:

Always `echo $CREATE_ARN` before running `create-integration` to visually verify the ARN is correct. The pattern must be: `arn:aws:lambda:REGION:12DIGITACCOUNTID:function:FUNCTIONNAME`

🔴 Issue 3 Hit: Manually created routes not picked up by CloudFormation-managed API Gateway stage

After fixing the ARNs and recreating integrations/routes manually, API Gateway still returned 500. CloudWatch logs remained empty. The routes were correctly wired and the Lambda permissions correct — but the stage was not serving the manually added routes.

Root cause:

When a CloudFormation-managed API Gateway stage has `AutoDeploy` enabled, it creates a "deployment" that is a snapshot of the API's routes at the time CloudFormation deployed it. Routes added manually afterwards are not automatically included in that deployment snapshot. The stage was still serving the old deployment (which had no routes at all) rather than the current state of the API.

Why this is a fundamental design conflict:

CloudFormation owns this API stage. Manually adding routes alongside CloudFormation-managed routes creates a conflict.

resources creates a state mismatch. CloudFormation thinks the API has no routes; reality has. The stage deployment reflects CloudFormation's view.

The correct fix — move Lambda functions, integrations, and routes INTO infrastructure.yaml so CloudFormation owns everything:

```
# This is the key architectural lesson:
# Resources created manually OUTSIDE CloudFormation are invisible to CloudFormation's
# deployment tracking. Add Lambda functions, integrations, and routes to the YAML template
# so everything is managed together and deployed atomically.

# After updating infrastructure.yaml, delete the manually created Lambda functions
# (CloudFormation cannot adopt externally created resources with same names):
aws lambda delete-function --function-name url-shortener-infra-createLink --region us-east-1
aws lambda delete-function --function-name url-shortener-infra-redirectLink --region us-east-1
aws lambda delete-function --function-name url-shortener-infra-adminLinks --region us-east-1

# Delete manually created routes and integrations too (they conflict):
aws apigatewayv2 delete-route --api-id $API_ID --route-id <id> --region $REGION
aws apigatewayv2 delete-integration --api-id $API_ID --integration-id <id> --region $REGION

# Then redeploy CloudFormation — it creates everything cleanly:
aws cloudformation deploy \
  --template-file infrastructure/infrastructure.yaml \
  --stack-name url-shortener-infra \
  --capabilities CAPABILITY_NAMED_IAM \
  --region us-east-1 \
  --parameter-overrides Environment=dev \
  --no-fail-on-empty-changeset

# After stack deploys, upload your actual code (CloudFormation uses placeholder code):
cd src/backend/functions
zip ../../../../build/create_link.zip create_link.py
zip ../../../../build/redirect_link.zip redirect_link.py
zip ../../../../build/admin_links.zip admin_links.py
cd ../../../../

aws lambda update-function-code --function-name url-shortener-infra-createLink \
  --zip-file fileb://build/create_link.zip --region us-east-1
aws lambda update-function-code --function-name url-shortener-infra-redirectLink \
  --zip-file fileb://build/redirect_link.zip --region us-east-1
aws lambda update-function-code --function-name url-shortener-infra-adminLinks \
  --zip-file fileb://build/admin_links.zip --region us-east-1
```

Key rule to remember:

Never mix manually created AWS resources with CloudFormation-managed resources in the same stack. Either manage everything through CloudFormation, or manage everything manually. Mixing them causes deployment tracking conflicts exactly like this.

Issue 4 Hit: CloudFormation EarlyValidation failure — ResourceExistenceCheck

```
Failed to create the changeset: Waiter ChangeSetCreateComplete failed:
Status: FAILED. Reason: The following hook(s)/validation failed:
[AWS::EarlyValidation::ResourceExistenceCheck]
```

Why it happened:

After updating `infrastructure.yaml` to include Lambda function definitions, CloudFormation tried to create the 3 Lambda functions by name. But those same function names already existed as manually created resources outside the stack. CloudFormation's `EarlyValidation` check (introduced in 2018) detects this before even starting deployment and refuses to proceed — it will not overwrite existing resources that it does not own.

Fix:

Delete the manually created Lambda functions first, then redeploy:

```
aws lambda delete-function --function-name url-shortener-infra-createLink --region us-east-1
aws lambda delete-function --function-name url-shortener-infra-redirectLink --region us-east-1
aws lambda delete-function --function-name url-shortener-infra-adminLinks --region us-east-1

# Then also delete manually created routes and integrations
# (they will conflict when CloudFormation tries to create identical ones):
aws apigatewayv2 get-routes --api-id $API_ID --region us-east-1 # get IDs
aws apigatewayv2 delete-route --api-id $API_ID --route-id <id> --region $REGION
aws apigatewayv2 get-integrations --api-id $API_ID --region us-east-1 # get IDs
aws apigatewayv2 delete-integration --api-id $API_ID --integration-id <id> --region $REGION

# Now redeploy cleanly:
aws cloudformation deploy \
  --template-file infrastructure/infrastructure.yaml \
  --stack-name url-shortener-infra \
  --capabilities CAPABILITY_NAMED_IAM \
  --region us-east-1 \
  --parameter-overrides Environment=dev \
  --no-fail-on-empty-changeset
```

🔴 Issue 5 Hit: Hardcoded old API ID inside `create_link.py` `shortUrl`

```
# Response showed wrong API ID in shortUrl:
{"shortUrl": "https://ijlmwfo9gd.execute-api.us-east-1.amazonaws.com/dev/testlink4"}
# But the actual working API is: bkewzxu3tc

# The bug in create_link.py – hardcoded old API ID:
'shortUrl': f"https://ijlmwfo9gd.execute-api.us-east-1.amazonaws.com/dev/{short_code}"
```

Why it happened:

`create_link.py` had the API Gateway base URL hardcoded as a string literal. When the project was first deployed it used a different API ID (`ijlmwfo9gd`). On re-deployment a new API Gateway was created with a new ID (`bkewzxu3tc`), but the old hardcoded URL in the code was not updated.

Note:

This only affects the cosmetic `shortUrl` field returned in the response — the redirect still works because the actual code is stored in DynamoDB and the redirect Lambda looks up by code, not by URL. This means the "copy link" button in the frontend gives users a broken URL.

Fix — use an environment variable instead of hardcoding:

```
# In create_link.py, replace the hardcoded shortUrl line with:
base_url = os.environ.get('BASE_URL', f"https://{event.get('requestContext', {}).get('domainName', '')}")
'shortUrl': f"{base_url}/{short_code}"

# Or simply use the request's own domain from the event (most robust):
domain = event.get('requestContext', {}).get('domainName', '')
stage = event.get('requestContext', {}).get('stage', 'dev')
'shortUrl': f"https://{domain}/{stage}/{short_code}"
```

Prevention:

Never hardcode API Gateway URLs or IDs in Lambda code. Use either environment variables (see [this](#)) or extract the domain dynamically from the Lambda event's requestContext.

✓ After all 5 issues were resolved, the successful test response was:

```
curl -X POST $API_URL/links \
  -H "Content-Type: application/json" \
  -d '{"url": "https://google.com", "customCode": "testlink4"}' \
  | python3 -m json.tool

# ✓ Response:
{
  "success": true,
  "shortCode": "testlink4",
  "shortUrl": "https://bkewzxu3tc.execute-api.us-east-1.amazonaws.com/dev/testlink4",
  "targetUrl": "https://google.com",
  "createdAt": "2026-03-23T11:07:04.821529Z"
}
```

2 Test Redirect endpoint

```
# Use curl -v (verbose), NOT curl -I (HEAD) – see issue below
curl -v $API_URL/testlink4 2>&1 | tail -20
```

WHAT TO LOOK FOR IN VERBOSE OUTPUT

You are checking: (1) `HTTP/2 301` status confirming `redirect_link.py` found the record. (2) `location: https://google.com` header confirming DynamoDB lookup returned the correct target. (3) `cache-control: no-cache` confirming the response headers are set correctly.

🚫 Issue Hit: curl -I gives 404 even when redirect is working

```
# This gave a misleading 404:
curl -I $API_URL/testlink4
# HTTP/2 404
```



```
# This correctly showed 301:
curl -v $API_URL/testlink4 2>&1 | tail -20
# HTTP/2 301
# location: https://google.com
```

Why it happened:

`curl -I` sends an HTTP HEAD request (not GET). The API Gateway route is defined as `GET /{code}` — it only matches GET requests. A HEAD request does not match GET, so API Gateway returns 404 "no matching route."

Rule to remember:

Always use `curl -v` (verbose GET) to test redirect endpoints. `curl -I` (HEAD) is misleading for routes defined as GET-only. The `-v` flag shows both headers and body so you get the full picture without the method mismatch problem.

3 Test Admin endpoint

```
curl $API_URL/admin/links | python3 -m json.tool
```

WHAT THIS CONFIRMS

Confirms `admin_links.py` can scan DynamoDB and compute statistics. Check that "links" contains your testlink record, that `click_count` incremented after the redirect test, and that `statistics.totalLinks` and `statistics.totalClicks` are correct. This exact data is what the admin dashboard will display.

4 Verify DynamoDB record directly

```
aws dynamodb scan \
  --table-name url-shortener-infra-links \
  --region us-east-1
```

WHY BYPASS LAMBDA AND CHECK DYNAMODB DIRECTLY?

This completely bypasses Lambda and API Gateway and talks directly to the database. It proves data was actually persisted. It shows the raw DynamoDB types (S = String, N = Number, BOOL = Boolean). Useful when you are unsure if a problem is in the API layer or the data layer — this isolates the database independently of everything else.

5 Open the frontend in a browser

Main app: `http://<bucket>.s3-website-us-east-1.amazonaws.com`

Admin: `http://<bucket>.s3-website-us-east-1.amazonaws.com/admin.html`

WHAT TO VERIFY IN THE BROWSER

Main page: paste a URL and click Shorten. You should get a real short link back with no "Demo Mode" notification. Click the short link — it should redirect to your URL. Open admin.html — it should show real data with your test links and click counts. If you see "Demo Mode" or fake data, the API_BASE_URL in app.js is wrong or the API is not responding — re-run curl tests 1–3 to isolate the layer that is failing.

7 DynamoDB Schema Reference

Attribute	Type	Role	Example
code	String	Primary Key (Hash Key) — all lookups go through this	abc123
target_url	String	The original long URL to redirect to	https://google.com
created_at	String	ISO 8601 timestamp of creation	2025-12-19T10:30:00Z
click_count	Number	Total redirects — atomically incremented on every redirect	42
custom_code	Boolean	True if user chose the code, false if auto-generated	false

KEY DESIGN DECISIONS TO EXPLAIN IN AN INTERVIEW

Why DynamoDB not PostgreSQL? URL shorteners have one access pattern: look up by short code. DynamoDB primary key lookup is $O(1)$ with single-digit millisecond latency — perfect for redirect performance. SQL is overkill, no joins needed, no complex queries needed.

Why on-demand billing? URL shorteners have highly variable traffic. On-demand means pay per request with automatic scaling. Provisioned mode requires predicting traffic upfront and over-provisioning.

Why ADD not SET for click_count? DynamoDB's atomic ADD is thread-safe. Two simultaneous redirects both calling ADD :1 will both succeed and count correctly. Two simultaneous redirects using SET (read-then-write) would both read "5", both write "6", and you would lose a count.

Why String for created_at not a date type? DynamoDB has no native datetime type. ISO 8601 strings sort lexicographically in the same order as chronologically — so date-range queries still work correctly even as plain strings.

8 API Endpoints Reference

Method	Path	Lambda	Success	Purpose
POST	/links	create_link.py	201	Shorten a URL with optional custom code
GET	/[{code}]	redirect_link.py	301	Redirect to target URL and increment click count
GET	/admin/links	admin_links.py	200	Return all links and statistics for the dashboard

create_link.py — What it does step by step

```
# 1. Parse JSON body from event['body']
# 2. Validate URL with urlparse – must have http or https scheme and a netloc (domain)
# 3. Check URL length is at most 2048 characters
# 4. If customCode: validate with regex ^[a-zA-Z0-9]{3,50}$ (3-50 alphanumeric chars)
# 5. If no customCode: generate random 6-char Base62 string (a-z A-Z 0-9 = 62^6 options)
#    Check for collision in DynamoDB, retry up to 5 times if collision
# 6. DynamoDB put_item with ConditionExpression='attribute_not_exists(code)'
#    If condition fails because code exists: return 409 Conflict
# 7. Return 201 with shortCode, shortUrl, targetUrl, createdAt
```

KEY POINTS TO REMEMBER

ConditionExpression='attribute_not_exists(code)' makes the write atomic. It says "only write this item if no item with this code already exists." Without this, two near-simultaneous requests generating the same random code could both succeed and one would silently overwrite the other. The condition check and write happen atomically in DynamoDB.

Base62 with 6 characters: 62^6 = about 56 billion possible codes. The probability of a collision on any single generation is 1 in 56 billion. The retry loop handles this astronomically rare case anyway.

redirect_link.py — What it does step by step

```
# 1. Extract short code from event['pathParameters']['code']
# 2. DynamoDB get_item(Key={'code': short_code})
# 3. If not found: return 404 HTML error page with user-friendly message
# 4. table.update_item: ADD click_count :1 (atomic thread-safe increment)
```

```
# Fire-and-forget: the 301 redirect is returned whether or not this update succeeds
# 5. Return 301 with Location: target_url and Cache-Control: no-cache
```

INTERESTING NUANCE — 301 VS 302 AND CACHE-CONTROL

301 = Permanent Redirect (browsers cache it). 302 = Temporary Redirect (not cached). 301 would mean after the first visit, the browser redirects directly without hitting the server — faster but means if you change the target URL in DynamoDB, cached users still go to the old destination. Adding Cache-Control: no-cache with a 301 actually disables the caching, making it behave like a 302 in practice. Worth flagging in an interview as a nuance.

admin_links.py — What it does step by step

```
# 1. Read query params: limit (default 50), lastKey (pagination cursor)
# 2. DynamoDB table.scan() – reads ALL items (no key filter)
#    Uses Limit and ExclusiveStartKey for pagination
# 3. convert_decimals(): DynamoDB returns numbers as Python Decimal type
#    json.dumps() cannot serialize Decimal – must convert to int or float first
# 4. Calculate statistics from the result set:
#    totalClicks = sum of all click_counts
#    averageClicks = totalClicks / len(links)
#    todayLinks = count of links where created_at starts with today's ISO date
# 5. Return links array + pagination + statistics
```

WHY SCAN() AND NOT QUERY()?

DynamoDB query() requires a partition key value — it fetches items with a known key. scan() reads the entire table regardless of key. For the admin dashboard where you want ALL links with no filter, scan() is correct. The trade-off is scan() reads every item and costs more for large tables. For a URL shortener at typical scale this is fine. At millions of records you would add a secondary index (GSI) to enable more efficient filtered queries.

9 Troubleshooting Quick Fixes

Problem

InsufficientCapabilitiesException

Root Cause

Template has a named IAM resource but CAPABILITY_IAM was used

Fix

Use `--capabilities CAPABILITY_NAMED_IAM`. The template has an explicit RoleName.

CloudFormation CREATE_FAILED	IAM permissions, name collision, or template syntax error	<code>aws cloudformation describe-stack-events --stack-name url-shortener-infra --query 'StackEvents[?ResourceStatus==`CREATE_FAILED`]'</code>
{"message": "Not Found"} from API	Route not created in API Gateway	Run Phase 2 Step 4 (create-integration + create-route). Verify: <code>aws apigatewayv2 get-routes --api-id \$API_ID</code>
403 Forbidden from API	Lambda invoke permission not granted to API Gateway	Re-run Phase 2 Step 5 (aws lambda add-permission) for all 3 functions
500 from API	Lambda code threw an exception	Check logs: <code>aws logs tail /aws/lambda/url-shortener-infra-createLink --follow</code>
CORS error in browser	Missing CORS config or OPTIONS route	HttpApi in CloudFormation has CORS set to AllowOrigins: *. Check it deployed correctly in the stack resources.
Frontend shows Demo Mode	API_BASE_URL wrong or API not responding	Update CONFIG.API_BASE_URL in app.js AND admin.js. Re-sync to S3. Test API with curl first.
DynamoDB ResourceNotFound	Lambda looking for wrong table name	Table is <code>url-shortener-infra-links</code> . Verify with <code>aws dynamodb list-tables</code> and check TABLE_NAME env var on Lambda.
S3 website returns 403	Bucket policy not deployed or wrong	CloudFormation deploys FrontendBucketPolicy with public s3:GetObject. Check it is in the stack resources with CREATE_COMPLETE status.
create-function fails already exists	Function exists from previous deployment	Use <code>aws lambda update-function-code</code> instead of create-function.

🔍 CloudWatch Logs — your best debugging tool for Lambda errors:

```
aws logs tail /aws/lambda/url-shortener-infra-createLink --follow
aws logs tail /aws/lambda/url-shortener-infra-redirectLink --follow
aws logs tail /aws/lambda/url-shortener-infra-adminLinks --follow
```

The --follow flag streams live logs. Press Ctrl+C to stop. Every print() statement in your Python code appears here. Full Python tracebacks for exceptions appear here too.

- ☐ **SETUP** AWS CLI installed — `aws sts get-caller-identity` returns your account ID

- ☐ **SETUP** Node 16+ and Python 3.9+ confirmed

- ☐ **SETUP** Repo cloned, `npm install` and `pip3 install boto3` done, in project root

- ☐ **INFRA** CloudFormation deployed with `CAPABILITY_NAMED_IAM` flag

- ☐ **INFRA** Stack outputs collected: `ApiGatewayId`, `ApiGatewayUrl`, `S3BucketName`, `DynamoDBTableName`, `LambdaRoleArn`

- ☐ **INFRA** DynamoDB table confirmed: `aws dynamodb list-tables`

- ☐ **INFRA** S3 bucket confirmed: `aws s3 ls | grep url-shortener`

- ☐ **BACKEND** 3 ZIP packages created in `/build`: `create_link.zip`, `redirect_link.zip`, `admin_links.zip`

- ☐ **BACKEND** `ROLE_ARN` variable populated from CloudFormation output

- ☐ **BACKEND** All 3 Lambda functions created (or updated) successfully

- ☐ **BACKEND** API Gateway integrations created for all 3 functions

- ☐ **BACKEND** 3 routes created: `POST /links`, `GET /{code}`, `GET /admin/links`

- ☐ **BACKEND** Lambda invoke permissions granted to API Gateway for all 3 functions

- ☐ **FRONTEND** `API_BASE_URL` updated in `app.js` with real API Gateway URL

- ☐ **FRONTEND** `API_BASE_URL` updated in `admin.js` with real API Gateway URL

- ☐ **FRONTEND** `DEMO_MODE: false` set in both files

- ☐ **FRONTEND** Files synced to S3 with correct content types for HTML, JS, CSS

- ☐ **TEST** `curl POST /links` returns 201 with `shortCode`

- ☐ **TEST** `curl -I GET /{code}` returns 301 with correct Location header

- ☐ **TEST** `curl GET /admin/links` returns links array and statistics

- ☐ **TEST** DynamoDB scan shows test record with incremented `click_count`

- ☐ **TEST** Frontend loads, creates a real link, admin dashboard shows live data with no Demo Mode banner

11 Talking Points for Interviewers

Why Serverless / Lambda?

No servers to provision or manage. Auto-scales from 0 to millions. Pay per invocation — free tier covers early traffic. Cold starts (~200ms) are the only trade-off, acceptable here since redirect traffic keeps functions warm.

Why DynamoDB over RDS?

URL shorteners have one access pattern: look up by code. DynamoDB primary key lookup is $O(1)$, single-digit milliseconds. No joins needed. On-demand billing handles variable traffic perfectly. SQL would be overkill.

Why API Gateway HTTP API not REST API?

HTTP APIs are about 71% cheaper and lower latency. We only need basic routing and CORS — no usage plans, API keys, or request transformation that REST API provides.

Why CloudFormation over manual setup?

Infrastructure as Code means reproducible, version-controlled, one-command deploy and teardown. No snowflake environments. Dev, staging, and production are guaranteed identical.

How does collision avoidance work?

Random 6-char Base62 = 56 billion possibilities. DynamoDB conditional write with `attribute_not_exists` makes it atomic. Up to 5 retries. Astronomically unlikely to need them at normal scale.

How does click tracking work?

DynamoDB atomic ADD — thread-safe, no lost counts under concurrent load. Fire-and-forget: redirect returns before the update completes so user latency is not affected.

Security considerations

IAM least-privilege roles. URL validation via `urlparse`. Custom code regex. Conditional DynamoDB writes prevent overwrites. CORS configured. No secrets in code — env vars for table names.

What would you add next?

Custom domain via Route 53 and CloudFront for HTTPS. Link expiry using DynamoDB TTL attribute. API key authentication. Rate limiting per user. DELETE endpoint. Bulk link creation via CSV upload.

🕒 Golden interview rule — narrate everything as you type:

Say out loud what you are doing and why at every step. "I am running CloudFormation first because it creates the DynamoDB table, IAM role, and API Gateway that Lambda needs — none of Phase 2 is possible without this." Silence looks nervous. Commentary shows mastery and architectural thinking. Interviewers want to see how you think, not just that you can type commands.

🕒 Time estimate with all tools pre-installed:

Setup 2 min → CloudFormation 3 min wait → Lambda deploy 3 min → Frontend 2 min → Testing 2 min → Total approximately 12 minutes. Practice until you can do it comfortably in under 15 minutes without referring to notes.

